```
                "application/x-www-form-urlencoded")    #A
            .uri(URI.create(    #A

"https://as.example.com/device_authorization"))   #A
            .POST(BodyPublishers.ofString(form))    #A
            .build();    #A
    var response = httpClient.send(request,
BodyHandlers.ofString());    #A

    if (response.statusCode() != 200) {    #B
        throw new RuntimeException("Bad response from AS: " +
    #B
            response.body());    #B
    }    #B
    return new JSONObject(response.body());    #C
}
```

#A Encode the client ID and scope as form parameters and POST them to the device endpoint.
#B If the response is not 200 OK then an error occurred.
#C Otherwise, parse the response as JSON.

The device that initiated the flow communicates the verification URI and user code to the user but keeps the device code secret. For example, the device might be able to display a QR code (figure 13.8) that the user can scan on their phone to open the verification URI, or the device might communicate directly with the user's phone over a local Bluetooth connection. To approve the authorization, the user opens the verification URI on their other device and logs in. They then type in the user code and can either approve or deny the request after seeing details of the scopes requested.